NUS National University of Singapore
Department of Statistics and Data Science
Faculty of Science

DSS5201 Data Visualisation

Week 2 Python Basics (Part I)

David CHEW

Semester 2 AY25/26

Last Week: Weekly Workflow

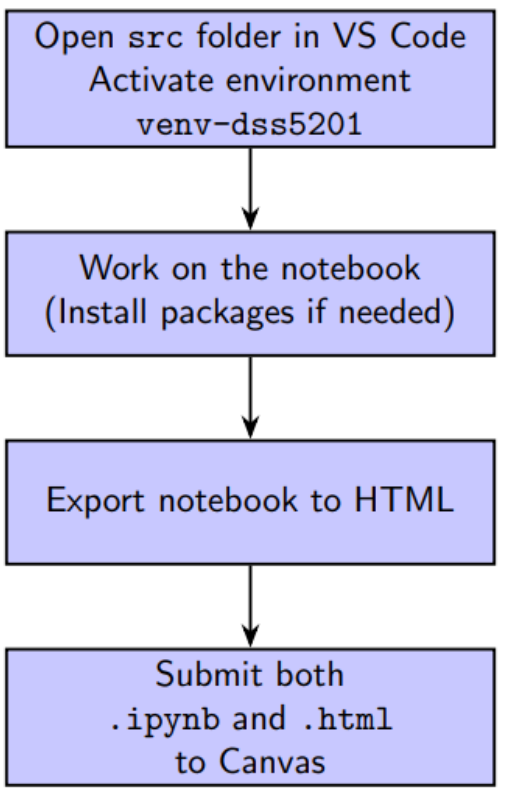
One time setup

Create virtual environment `venv-dss5201`

Activate environment

Install packages with `requirements.txt`

Weekly workflow



Windows: `venv-dss5201\Scripts\activate`
macOS: `source venv-dss5201/bin/activate`

Select kernel: `venv-dss5201`

```
jupyter nbconvert --to html --execute notebook_name.ipynb
```

Replace with actual filename of notebook

This Week: Python Basics

- Python Objects
- Data Types
- Data Structures
- Conditional Expressions

Python Objects

Python Objects

Everything in Python is an **object** and every object has a **type** (class).

Understanding the type of an object helps

- Determine what methods and attributes are available for that object, and
- How it will behave in our code.

Naming Objects

To store data or results, we create an object by assigning a name to it.

- Names can include letters, digits, and underscores.
- Cannot start with a digit: `5201dsa` is not valid.
- Case sensitive: `avg`, `Avg`, `AVG` refer to different objects.

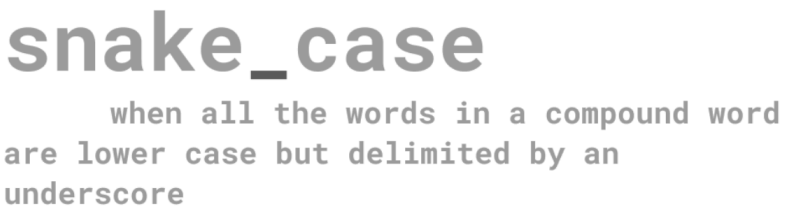
Naming Objects: Reserved Words

Python reserved words (keywords) cannot be used as variable names

- Here are a few examples:

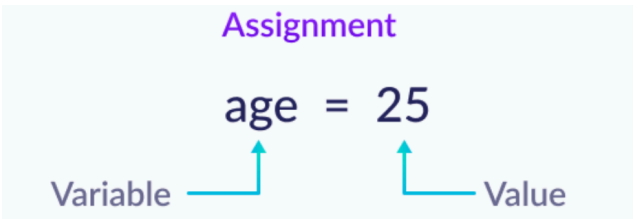
False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
asynch	elif	if	or	yield

Naming Objects: Explicit is better than implicit!



1. Use descriptive names.
    - Choose names that describe the purpose of the variable or function.
    - Example: Instead of `x`, use `temperature` or `age`.
2. Use underscores for readability (snake case).
    - Use **underscores** to separate words for better readability.
    - Example: Use `avg_temperature` rather than `avgtemperature`.

Assignment Operator



To create a new object, we assign a value using the **assignment operator** (=).

- Think of it as putting some information into a labelled box.

Other Assignment Operators

Another assignment operator that's commonly used is `+=`.

- Adds a value to the current value of the variable and updates it.

```
a = 42
a += 1       # same as a = a + 1
print(a)
```

```
43
```

- It is commonly used in loops for counting and accumulating sums.

Data Types

Data Types

In Python, variables can be of different types (class).

Throughout the semester, we will encounter several **common data types**:

- **Integers** (`int`): whole numbers.
- **Numeric** (`float`): numbers that contain a decimal.
- **Boolean** (`bool`): data that take on the value of either `True` or `False`.
- **String** (`str`): data that are used to represent strings of text.
  - A special string type in `pandas` is called **categorical**.
- **Datetime** (`datetime`): date and time information.

Example

- Floating-point (`float`) numbers contains a decimal.
- We assign a whole number `42` to `a`, and a float `42.7` to `b`.
- Check the type of any object with `type()`.

```
1  a = 42
2  type(a)
```

```
int
```

```
1  b = 42.7
2  type(b)
```

```
float
```

Coercion Functions

- `type()` finds out the type of an object.
- We can also convert an object to a different type:

Type	Coercing
numeric	`float(var)` or `int(var)`
logical	`bool(var)`
character	`str(var)`
categorical	`pd.Categorical(var)`, with the `pandas` package
datetime	`datetime.strptime(var)`, from `datetime`

Example

- `float()` converts a data type to a float.
- `int()` converts a data type into an integer. Any fractional part is removed.
- `str()` converts a data type to a string.

```
1  float(42)
```

```
42.0
```

```
1  int(42.7)
```

```
42
```

```
1  str(42.7)
```

```
'42.7'
```

Example

When converting strings to numbers, make sure the string is numeric

```
1  int('123')
```

```
123
```

Otherwise we will get an error.

```
1  int('abc')        # ValueError
```

Basic Arithmetic

Basic arithmetic operations in Python:

- Addition: `1 + 2 + 3`
- Subtraction: `5 - 2`
- Multiplication: `2 * 3.14`
- Exponentiation: `2 ** 3`
- Division: `100 / 3`

Division Operators

- True division (`/`) returns a float.
- Floor division (`//`) returns the integer part of the result.
- The modulo operator (`%`) returns the remainder.

```
1  100 / 3
```

```
33.333333333333336
```

```
1  100 // 3
```

```
33
```

```
1  100 % 3
```

```
1
```

The `math` Standard Library

We can do basic arithmetic operations with Python's built-in operators.

- For more advanced functions, we need the `math` standard library.

```
1 import math
```

- Here are some example functions:

Operation	Example
Square root	`math.sqrt(16)`
Rounding	`math.floor(2.7)` and `math.ceil(2.7)`
π	`math.pi`
Euler's number	`math.exp(1)`
Natural logarithm	`math.log(10)`
Base-10 logarithm	`math.log10(10)`

Rounding

- We can round up (down) with `math` functions:
- To round a number to decimal places, we can just use the built-in function `round()`.

```
1  x = math.pi
2  print(math.ceil(x))
```

```
4
```

```
1  print(math.floor(x))
```

```
3
```

```
1  print(round(x, 5))
```

```
3.14159
```

Strings

- A **string** is a sequence of characters enclosed in matching single or double quotes.
- We can include double quotes in a string with matching single quotes, and vice versa.
- `len()` returns the length of a string.

```
1 str1 = 'hello'
2 type(str1)
```

```
str
```

```
1 len(str1)
```

```
5
```

String Methods

Python provides many useful string methods:

- Convert to lower or upper case with `lower()` or `upper()`:
- Check whether a string starts or ends with a given character, with `startswith()` or `endswith()`:

```
1 str1.upper()
```

```
'HELLO'
```

```
1 str1.startswith('h')
```

```
True
```

Example

- We can `split()` a string into words by white spaces. It returns a list of individual words:
- To remove white space(s) from a string, we can use `strip()`. This is useful for cleaning messy data entries before matching or comparing values.

```
1 str2 = 'Is this a Wednesday?'
2 str2.split()
```

```
['Is', 'this', 'a', 'Wednesday?']
```

```
1 str3 = '   Clementi  '
2 str3.strip()
```

```
'Clementi'
```

Formatted Strings

A **formatted string** (f-string) is a string that is prefixed with `f`.

- Inside an f-string, we can insert an object with curly brackets.
- We can also format how numbers are displayed:

```
1  x = math.pi
2  print(f'{x:.2f}')
```

```
3.14
```

```
1  sales = 1000000
2  print(f'{sales:,}')
```

```
1,000,000
```

Example

- We've also seen f-strings in `week1_lecture.ipynb`.
- The following returns the Python version in the current environment.

```
1 import sys
2 print(f'Python version: {sys.version}')
```

```
Python version: 3.14.2 (v3.14.2:df793163d58, Dec  5 2025, 12:18:06) [Clang 16.0.0
(clang-1600.0.26.6)]
```

Date & Time

Python provides a `datetime` standard library to handle dates and times.

```
from datetime import datetime
today = datetime.today()
print(f'The current date and time is {today}.')
```

```
The current date and time is 2026-01-20 12:00:04.548455.
```

- We can use `timedelta` to add or subtract time from a day.

```
from datetime import timedelta
next_week = today + timedelta(days = 7)
print(next_week)
```

```
2026-01-27 12:00:04.548455
```

Full / Absolute Path

We can use the `os` standard library to show the full path in Python.

```python
import os
cwd = os.getcwd()
print(f'Current working directory (full path) is {cwd}.')
```

```
Current working directory (full path) is d:\dss5201\src.
```

- `getcwd()` returns the **full (absolute) path** where our notebook is running.
  - A full path works only on your computer.
- In this semester, we will use **relative path** so your code runs the same way on your computer, your team mates', and the teaching team's computers.

Example

- Recall our standard folder structure:

```
DSS5201
  |--- data
  |--- src      # current working directory
```

- `path.dirname()` returns the folder one level up from a given directory.

```
parent_dir = os.path.dirname(cwd)
print(f'The parent directory is {parent_dir}.')
```

```
The parent directory is .
```

Boolean

A Boolean (`bool`) variable has two possible values: `True` or `False`.

- **Comparison operators** return a boolean result:

Operator	Description
`x == y`	is `x` equal to `y`?
`x != y`	is `x` not equal to `y`?
`x > y`	is `x` greater than `y`?
`x >= y`	is `x` greater or equal to `y`?
`x < y`	is `x` smaller than `y`?
`x <= y`	is `x` smaller or equal to `y`?

- **Membership operators** check if something is found in an object:

Operator	Description
`x in y`	is `x` found in `y`?
`x not in y`	is `x` not found in `y`?

Example

- Combining conditions:

```
score = 85
score > 80 and score < 90
```

```
True
```

- Working with strings:

```
city = "Singapore"
city == "Tokyo"
```

```
False
```

```
"Sing" in city
```

```
True
```

Summary I

- Everything is an object with a type.
- We assign value(s) to an object with `=` and display type with `type()`.
  - Common types: `int`, `float`, `str`, `bool`, and `datetime`.
  - Functions for type conversion.
- Basic arithmetic operations.
- Basic string methods and f-strings.
- Boolean values from comparisons (`==`, `<`, etc) and membership checks (`in`, `not in`).

Data Structures

Basic Data Structures

The main objects in Python that contain data are

- Lists: Defined with `[]`, mutable.
- Tuples: Defined with `()`, immutable.
- Dictionaries: Defined with `{}`, key-item pairs and mutable.

Very soon, we shall see that the more common objects we shall deal with are DataFrames (from `pandas`) and arrays (from `numpy`) – these require additional packages.

List and Tuples

List are defined with (square) brackets `[]`.

- Values are separated by commas.
- Mutable – we can change, add, or remove elements.

Here we create a list of length 3, with values 1, 2, and 3.

```
1 list1 = [1, 2, 3]
2 print(list1)
```

```
[1, 2, 3]
```

List

- A list can contain a combination of types.

```
1 list2 = [1, 'hello', True, 3.14]
2 print(list2)
```

```
[1, 'hello', True, 3.14]
```

- We can find out the length of the list with `len()`:

```
1 len(list2)
```

```
4
```

Zero-Indexed

Python is a zero-indexed language. The index begins from 0 instead of 1.

- To access the first element in `list2`:

```
1  list2[0]
```

```
1
```

- Access the last element with negative indexing:

```
1  list2[-1]
```

```
3.14
```

- An `IndexError` will occur when the index specified is out of range.

```
1  list2[9]              # IndexError
```

Lists are Mutable

Lists are mutable. We can amend values in a list.

```
1  list2[0] = False
```

Now `list2` becomes

False	hello	True	3.14
0	1	2	3

Tuples

Tuples look similar to lists, but they are defined with parentheses `()`.

- Values are also separated with commas.
- Immutable – cannot be changed after creation.

```
1  tuple1 = (1, 'hello', True, 3.14)
2  tuple1
```

```
(1, 'hello', True, 3.14)
```

```
1  type(tuple1)
```

```
tuple
```

Tuples are Immutable

Tuples are immutable. The following will raise a `TypeError`.

```
tuple1[2] = False        #TypeError
```

- Because tuples cannot be modified, they generally use less memory.
- It offers faster performance for creation and access – beneficial for large data sets.
- Example: Fixed data like geographic coordinates.

```
tuple2 = (1.3521, 103.8198)
tuple2
```

```
(1.3521, 103.8198)
```

Accessing Multiple Elements

To slice multiple elements, we use the syntax `name[a:b]`, where `a` and `b` are integers.

- Slicing includes the first (left) value and excludes the second (right) value.
- So `name[a:b]` would return the elements at indices `a, a+1, ... , b-1`.

Example

```
1 list3 = ['p', 'y', 't', 'h', 'o', 'n']
2 list3
```

```
['p', 'y', 't', 'h', 'o', 'n']
```

- Access the first element:

```
1 list3[0]
```

```
'p'
```

- Access the first to the third elements:

```
1 list3[0:3]
```

```
['p', 'y', 't']
```

- Slice indices are optional. We can omit the start or end index:

```
# From the beginning up to (but not including index 3)
list3[:3]
```

```
['p', 'y', 't']
```

```
# From index 3 to the end of the list
list3[3:]
```

```
['h', 'o', 'n']
```

- We can also specify a step size to skip elements:

```
# Select every other value in the list
list3[:: 2]
```

```
['p', 't', 'o']
```

List Methods

We can use built-in methods to modify a list:

- Defined on the list object and accessed with the dot notation (`.`)
- `append()` adds an element to the end:

```
1  list3.append('new item')
2  list3
```

```
['p', 'y', 't', 'h', 'o', 'n', 'new item']
```

- `remove()` deletes the first matching element by value.

```
1  list3.remove('n')
2  list3
```

```
['p', 'y', 't', 'h', 'o', 'new item']
```

Dictionaries

Data are stored in **key-value** pairs in a **dictionary**.

- Defined with braces (“curly brackets”).
- The key is a unique identifier; item can be of any data type.
- A key and associated value are separated by a colon, and key-value pairs are separated by commas.

```
dict1 = {
    'A': 5201,
    'B': ['Stats', 'Data Science'],
    'C': [True, False, True]}
```

Dictionaries: Accessing Keys and Values

To obtain all keys in dictionaries, we use the `keys()` method:

```
1  dict1.keys()
```

```
dict_keys(['A', 'B', 'C'])
```

To obtain all the values, use `values()`:

```
1  dict1.values()
```

```
dict_values([5201, ['Stats', 'Data Science'], [True, False, True]])
```

Dictionaries: Accessing specific values

There are two common ways to access values by its key:

- Use square brackets with the key inside:

```
1 dict1['B']
```

```
['Stats', 'Data Science']
```

- Use the `get()` method:

```
1 dict1.get('B')
```

```
['Stats', 'Data Science']
```

Dictionaries are Mutable

Dictionary items can be added, updated, or deleted:

- To modify a specific value by its key:

```
dict1['A'] = 5101
dict1
```

```
{'A': 5101, 'B': ['Stats', 'Data Science'], 'C': [True, False, True]}
```

- To delete a key-value pair:

```
del dict1['C']
dict1
```

```
{'A': 5101, 'B': ['Stats', 'Data Science']}
```

Recap: Brackets

We've introduced several types of "brackets". They play different roles depending on the context.

The most common uses are:

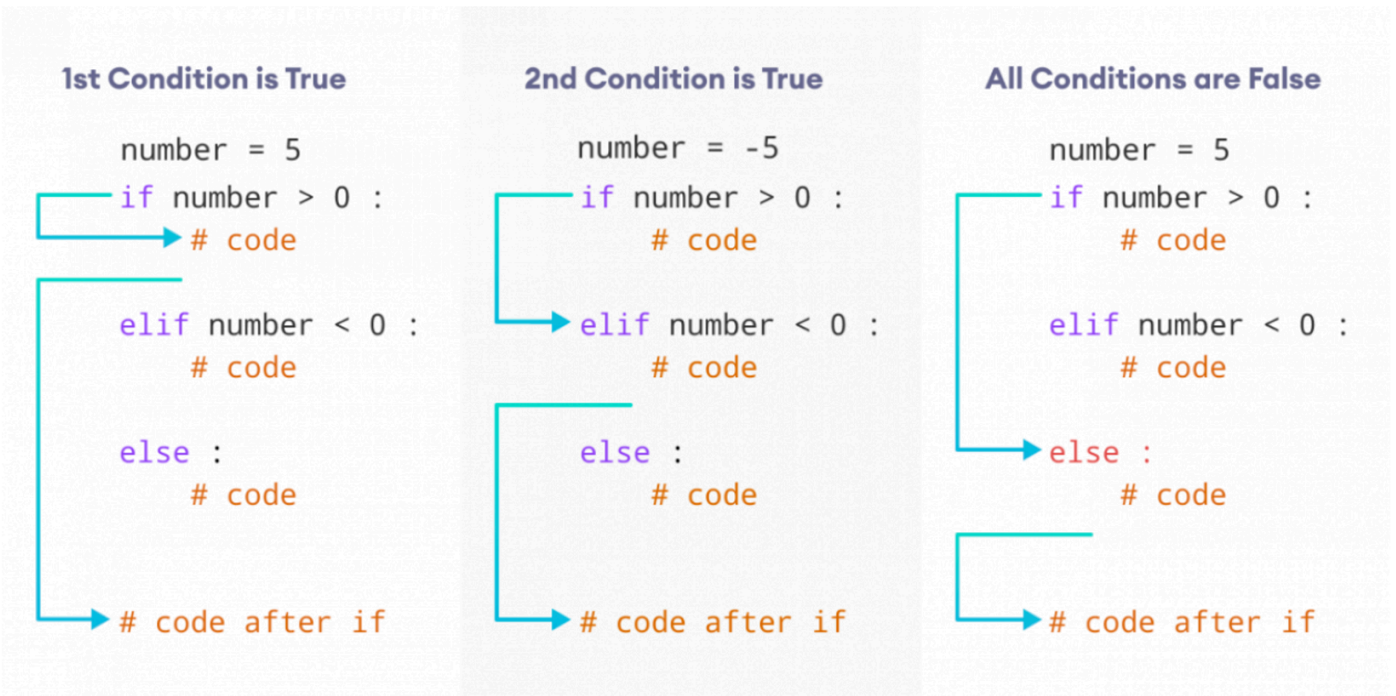
- `[]` Brackets are used to denote a list, or to access a position with an index.
  - `var[0]` gets the first element of a variable named `var`.
- `()` Parentheses are used to denote a tuple or arguments of a function.
  - `type(x)` where `x` is the input passed the the function `type()`.
- `{}` Braces are used to denote a set or a dictionary.
  - `{'first_name': 'a', 'last_name': 'b'}`.

Conditional Expressions

Control flow

Conditional logics are essential for controlling the flow of a program.

- `if` statement is used to execute a block of code only if a specified condition is true.
- Alternatively, we can use an `else` block to execute code if the condition is false.
- We can use `elif` (short for `else if`) to check multiple conditions sequentially.



Whitespace and indentation

Let's look closer at the code.

- Begins with `if`, followed by a boolean expression and a colon.
- The code block under `if` **must be indented**. The standard practice is to use 4 spaces.
- `elif` and `else` are aligned with the `if` statement. Their code blocks must also be indented.

```
number = -5
if number > 0 :
    # code

elif number < 0 :
    # code

else :
    # code
```

Whitespace and **indentation** are important in Python – part of the syntax and not optional.

Example: `if`

```
w = -5
if w > 0:
    print('positive')
```

The condition is not met, so the `print()` statement is not executed.

Example: `else`

An `else` statement contains a body of statements that is executed when the `if` statement condition is `False`.

```
w = -5
if w > 0:
    print('positive')
else:
    print('negative or zero')
```

```
negative or zero
```

Example: `elif`

We can use `elif` to check multiple conditions.

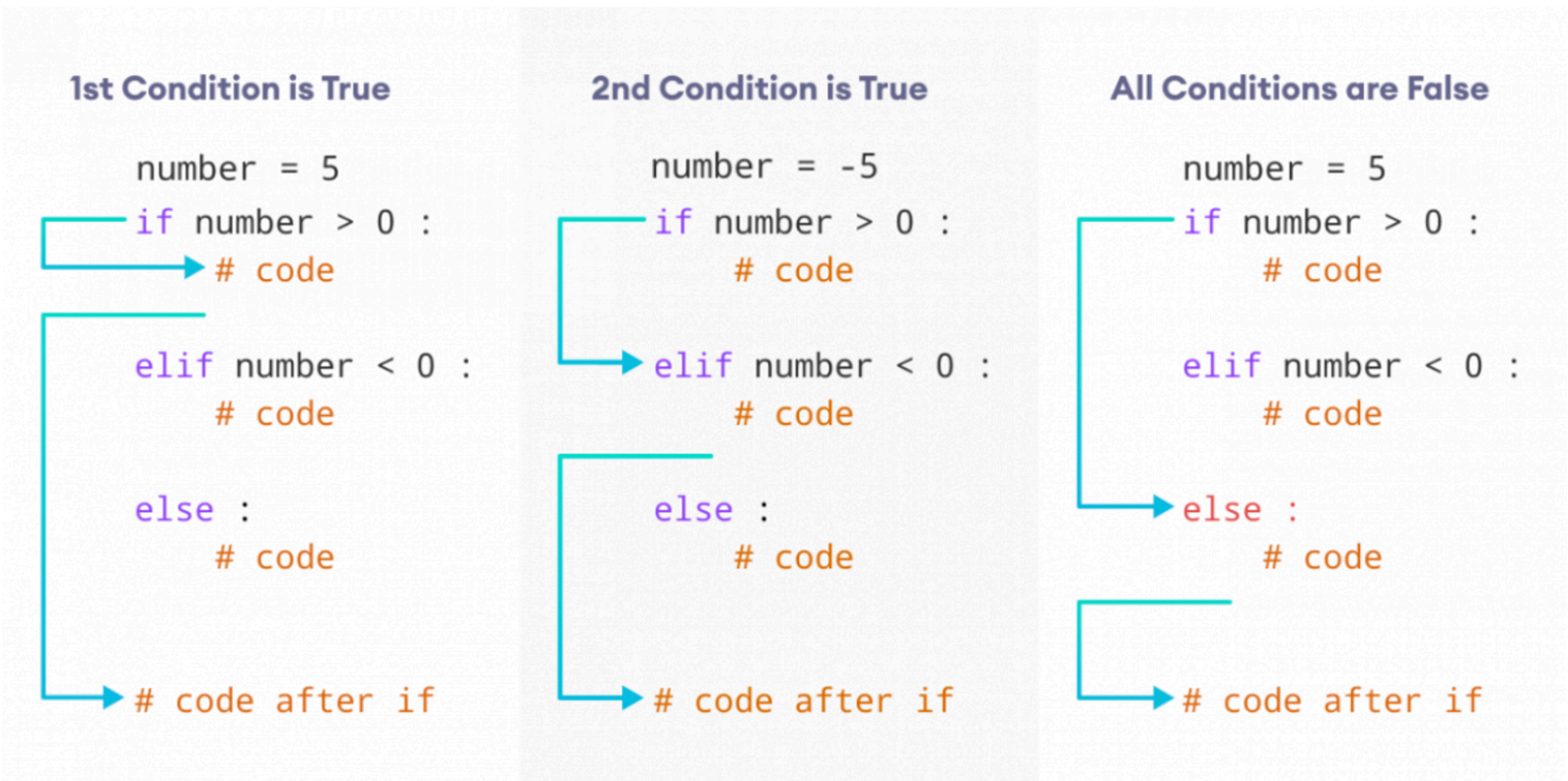
```
w = -5
if w > 0:
    print('positive')
elif w < 0:
    print('negative')
else:
    print('zero')
```

```
negative
```

`else` catches anything that's not caught by the previous conditions.

Recap: Control Flow

- `if` executes a code block only if a specified condition is true.
- `else` can be used to execute code if the condition is false.
- `elif` can be added to check multiple conditions sequentially.



NumPy's np.where()

We can express the same logic with NumPy's where() function.

- A simple if-else condition:

```
import numpy as np
w = -5
result = np.where(w > 0, 'positive', 'negative or zero')
print(result)
```

```
negative or zero
```

- For multiple conditions, we can use nested np.where(), though it becomes less readable as the logic becomes more complex.

```
result = np.where(w > 0, 'positive',
                  np.where(w < 0, 'negative', 'zero'))
print(result)
```

```
negative
```

NumPy's `np.select()`

For more complex logic, `NumPy`'s `select()` function is a better alternative.

- Easily handles multiple conditions and avoids nesting.
- It evaluates each `condition` in order and returns the values from `choices`.
- If none of the conditions are met, it returns the `default`.

```
w = -5
conditions = [w > 0, w < 0]
choices = ['positive', 'negative']
result = np.select(conditions, choices, default = 'zero')
print(result)
```

```
negative
```

Summary II

Basics in Python Programming

- Data types and data structures
- Conditional expressions: `if-elif-else`
- Standard libraries: `math`, `sys`, `datetime`, `os`

To Do:

- Make sure you've joined our DataCamp Classroom.
- The first DataCamp assignment is due by **Week 3 Friday 2359 hrs**.

Next Week:

- More on Python Basics – `NumPy` and `pandas`.